

Product API Reference

How getProducts works — everything you need to build the Product Details Page

The Basics

All product data on WaltonPlaza comes from a single GraphQL query called getProducts. You hit it with a POST request, pass some filters, and get back product details.

	Details
Endpoint	https://devapi.waltonplaza.com.bd/graphql
Method	POST
Content-Type	application/json

Query Structure

Here's a full query that fetches everything you'd need for a product detail page. The comments tell you which UI section each field maps to:

```
{
  getProducts(
    pagination: { skip: 0, limit: 10 }
    filter: {
      uid: "P-4TCF9V"          # or use posItemCode: "25311"
    }
  ) {
    message
    statusCode
    result {
      count
      products {
        uid
        enName                  # Product title

        images {                # Image gallery
          url
        }

        productAttributes {      # → Basic Information tab
          enLabel
          values { enName }
        }
      }
    }
  }
}
```

```
    detailedDescriptions { # → Detailed Information tab
      enLabel
      values { enName }
    }

    deliveries { # → Terms & Conditions tab
      enLabel
      values { enName }
    }

    serviceAndDeliveries { # → Warranty Information tab
      enLabel
      values { enName }
    }

    priceAndStocks { # → Special Feature section
      enLabel
      values { enName }
    }

    variants { # → Price & variant selector
      mrpPrice
      ebsItemCode
      posItemCode
      quantity
      discount {
        amount
        value
        type
      }
    }
  }
}
}
```

How Filters Work

You can filter products by different identifiers. Pass whichever one you have:

Filter	Example	When to Use
uid	"P-4TCF9V"	This is the main product ID. Use this when navigating from a product listing page.
posItemCode	"25311"	POS system code. Useful when you're linking from inventory/ERP systems.

isActive	null / true / false	Set to null to get all products regardless of status. true = active only.
----------	---------------------	---

Pagination is straightforward — skip is the offset, limit is how many you want back.

What Goes Where on the Product Page

This is the most important part. Here's how each API field maps to sections on the Product Detail Page:

API Field	UI Section	What's Inside
enName	Product title / heading	English product name
images	Image gallery / carousel	Array of image URLs
productAttributes	Basic Information tab	Brand, model, color, etc.
detailedDescriptions	Detailed Information tab	Extended specs & features
deliveries	Terms & Conditions tab	Delivery terms, return policy
serviceAndDeliveries	Warranty Information tab	Warranty period, service info
priceAndStocks	Special Feature section	Highlighted features, badges
variants	Price display / Buy section	Price, discount, stock per variant

Notice the pattern: productAttributes, detailedDescriptions, deliveries, serviceAndDeliveries, and priceAndStocks all share the same structure — an array of { enLabel, values: [{ enName }] }. So you can build one reusable component to render all of them.

Variants & Pricing

Each product can have multiple variants (think different colors or storage options). Each variant has its own price and stock:

Field	What It Is
mrpPrice	Original price (MRP) — this is the "was" price you'd show with a strikethrough
discount.amount	The discount number — could be a flat BDT amount or a percentage, depends on type
discount.value	Final selling price after discount — this is what the customer actually pays

discount.type	Either "flat" (discount.amount is BDT off) or "percentage" (discount.amount is % off)
quantity	Stock available. If 0 → show "Out of Stock"
ebsItemCode	Internal EBS system code (you probably won't display this to customers)
posItemCode	POS system code (internal use)

Price Math

Here's how to calculate the selling price yourself if you need to verify:

```
// If type is "flat":  
sellingPrice = mrpPrice - discount.amount  
  
// If type is "percentage":  
sellingPrice = mrpPrice - (mrpPrice × discount.amount / 100)  
  
// discount.value should already equal the selling price.  
// Use it directly for display, but the formula above  
// helps you build the "Save ₳X" or "X% OFF" badge.
```

Quick example — a product with mrpPrice: 45,000 and discount: { amount: 15, type: "percentage" }:

```
sellingPrice = 45000 - (45000 × 15 / 100) = 45000 - 6750 = 38,250  
discount.value would be 38250  
You'd show: ₳38,250 (₳45,000 strikethrough) 15% OFF badge
```

Things to Watch Out For

A few gotchas you'll run into when working with this API:

- discount can be null — means no discount on that variant. Just show mrpPrice as the selling price, no strikethrough or badge.
- images can be an empty array — you'll need a placeholder image.
- quantity: 0 means out of stock. Disable the buy button and show a clear indicator.
- Some sections like serviceAndDeliveries or productAttributes can come back as null or an empty array — hide the tab or show a "No information available" message.
- Always check statusCode in the response. 200 means success. Anything else — show an error state and use the message field for context.
- The enLabel and enName fields are in English. These are what you display as labels and values in the UI.

Quick Example: Fetching in JavaScript

Here's a minimal fetch call so you can see how it fits together:

```
const response = await fetch(
  "https://devapi.waltonplaza.com.bd/graphql",
  {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      query: `{
        getProducts(
          filter: { uid: "P-4TCF9V" }
          pagination: { skip: 0, limit: 1 }
        ) {
          message
          statusCode
          result {
            products {
              enName
              images { url }
              variants {
                mrpPrice
                quantity
                discount { amount value type }
              }
            }
          }
        }`
      })
    })
  );

const data = await response.json();
const product = data.data.getProducts.result.products[0];
```

That's it. The dev API is open for testing — try different UIDs and explore the responses in Postman or your browser's DevTools. If the query runs and you can see the data, you're good to start building.

Good luck!